Python-Pandas

Introduction to Pandas

What is Pandas?

- *A Python library for data analysis and manipulation.*
- *Built on top of NumPy.*

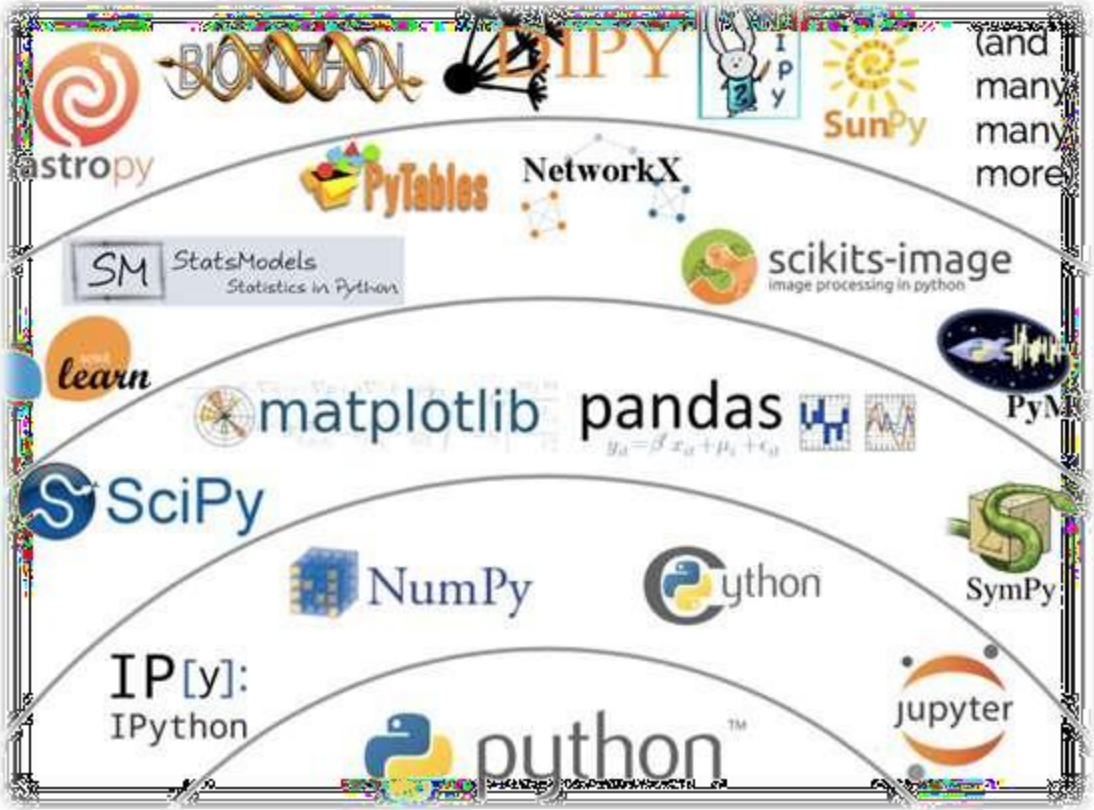
Why Use Pandas?

- Easy to handle missing data.
- Efficient for data wrangling.
- Supports operations for structured data.

Artificial Intelligence
Machine Learning
Deep Learning

Agenda

- Introduction – Panel Data System
- History and usage
- Series, DataFrame, Panel
- Basic Operations



One guiding principle of Python code is that
“explicit is better than implicit”

Introduction

- Pandas is Python package for data analysis.
- Pandas is an open-source Python Library built on top of Numpy
- Provides high-performance data manipulation and analysis tool using its powerful data structures.
- The name Pandas is derived from the word Panel Data – an Econometrics from Multidimensional data.
- In 2008, developer Wes McKinney started developing pandas when in need of high performance, flexible tool for analysis of data. {at AQR capital Management LLC}

- 30,000 lines of tested Python/Cython code

Introduction

```
import pandas
pandas.__version__
```

```
'0.20.3'
```

- Pandas can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data —

  load → prepare → manipulate → model → analyze.

- Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.
- Pandas is easy to use and powerful, but

  "*with great power comes great responsibility*"

eases data munging

Numpy vs Pandas

- Pandas is designed for working with tabular or heterogeneous data.
- NumPy, is best suited for working with homogeneous numerical array data
- **Numpy** is required by **pandas** (and by virtually all numerical tools for Python)
- numpy consumes (roughly 1/3) less memory compared to pandas
- numpy generally performs better than pandas for 50K rows or less
- pandas generally performs better than numpy for 500K rows or more
- for 50K to 500K rows, it is a toss up between pandas and numpy depending on the kind of operation

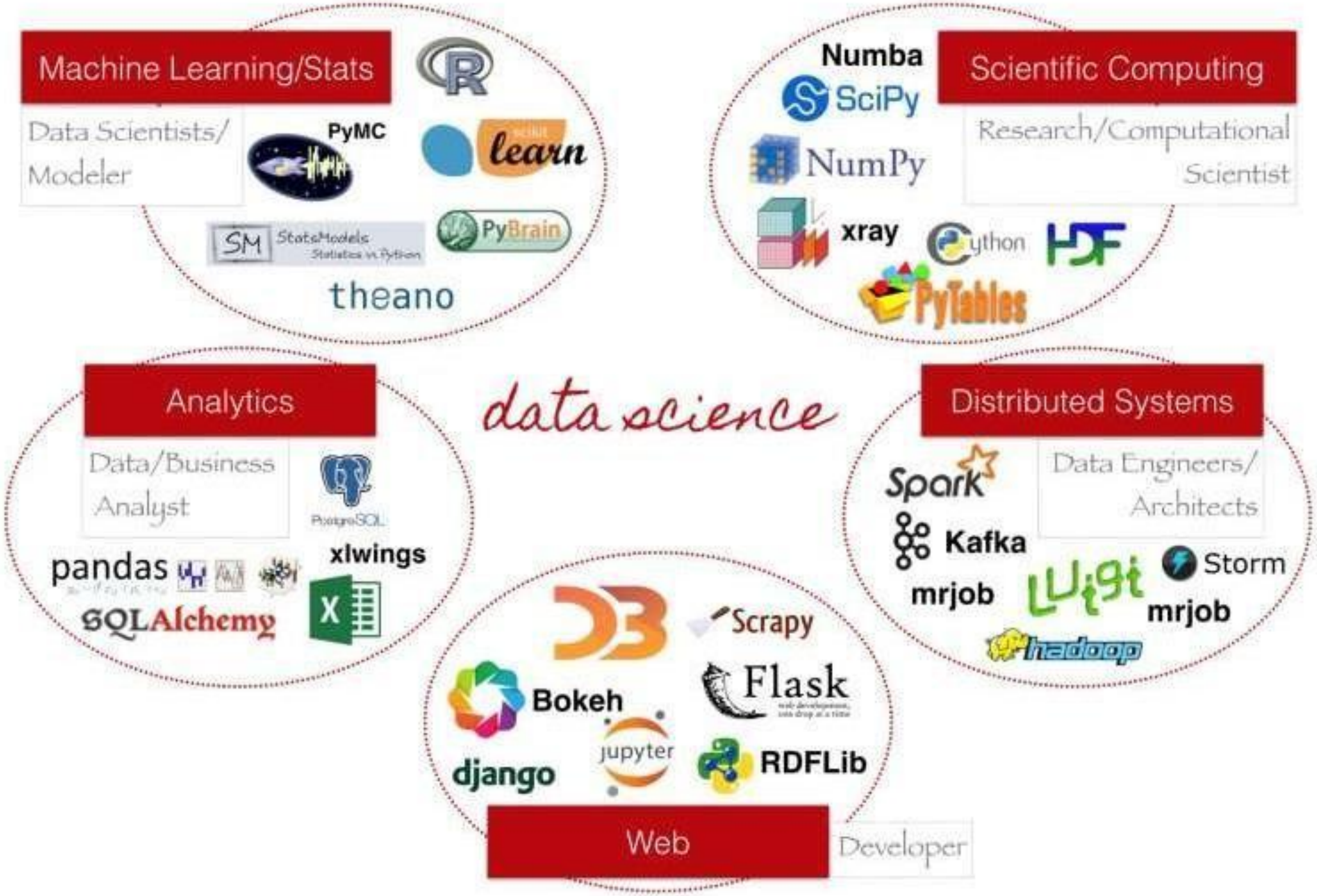
✓ Pandas became an open source project in 2010

✓ Now has 800 distinct contributors in developer community

Source:gouthamanbalaraman.com

Why pandas

- Big part of Data Science is Data Cleaning.
- Pandas is a power tool for data cleaning



Key Features of Pandas

- Fast and efficient DataFrame object with default and customized indexing.
- Tools for loading data into in-memory data objects from different file formats.
- Data alignment and integrated handling of missing data.
- Reshaping and pivoting of date sets.
- Label-based slicing, indexing and subsetting of large data sets.
- Columns from a data structure can be deleted or inserted.
- Group by data for aggregation and transformations.
- High performance merging and joining of data.
- Time Series functionality.

Source : tutorialpoint.com

Pandas and NumPy

- Pandas and NumPy both hold data
- Pandas has column names as well
- Makes it easier to manipulate data

Pandas NumPy Scikit-Learn workflow

- Start with CSV
- Convert to Pandas
- Slice and dice in Pandas
- Convert to NumPy array to feed to scikit-learn
- NumPy is faster than Pandas
- Both are faster than normal Python arrays

Data Structures

- Pandas deals with the following three data structures –
  Series , DataFrame , Panel
- These data structures are built on top of NumPy array, which means they are fast.
- DataFrame is a container of Series, Panel is a container of DataFrame.

Data Structure	Dimensions	Description
Series	1	1D labeled homogeneous array, size immutable.
Data Frames	2	General 2D labeled, size-mutable tabular structure with potentially heterogeneously typed columns.
Panel	3	General 3D labeled, size-mutable array.

Installing and Importing Pandas

- **Installation**

```
pip install pandas
```

- **Importing Pandas**

```
import pandas as pd
```

Pandas Data Structures

Series (1D data)

- A one-dimensional labeled array.

DataFrame (2D data)

- A two-dimensional, size-mutable, tabular data structure.

Creating a Series

- **From a List**

```
import pandas as pd

data = [10, 20, 30, 40]
s = pd.Series(data)
print(s)
```

- **Custom Index**

```
s = pd.Series(data, index=['a', 'b', 'c', 'd'])
print(s)
```

Creating a DataFrame

- **From a Dictionary**

```
import pandas as pd
data = {
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35],
'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
print(df)
```

Reading and Writing Data

- **Reading a CSV File**

```
df = pd.read_csv('data.csv')
print(df)
```

- **Writing to a CSV File**

```
df.to_csv('output.csv', index=False)
```

Viewing Data

- **Viewing Rows and Columns**

```
print(df.head()) # First 5 rows
print(df.tail(3)) # Last 3 rows
```

- **DataFrame Info**

```
print(df.describe())
print(df.info())
```

Rename Columns

```
df =
pd.DataFrame({"A": [1, 2], "B": [3, 4], "C": [5, 6]})

# Rename specific columns inline
df.rename(
columns={"A": "Alpha", "B": "Beta"}, inplace=True
)
```

Selecting Data

- **Selecting Columns**

```
print(df['Name'])
print(df[['Name', 'Age']])
```

- **Selecting Rows**

```
print(df.iloc[0]) # By index
print(df.loc[0]) # By label
print(df.iloc[1:3]) # Slice rows
```

Data Manipulation

- **Adding a Column**

```
df['Salary'] = [50000, 60000, 70000]
```

- **Filtering Rows**

```
filtered_df = df[df['Age'] > 30]
```

- **Dropping Columns or Rows**

```
df = df.drop('Salary', axis=1) # Drop column
df = df.drop(0, axis=0) # Drop row
```

Aggregation and Grouping

- **Aggregations**

```
print(df['Age'].mean())
print(df['Age'].sum())
```

- **Grouping**

```
grouped = df.groupby('City')
print(grouped['Age'].mean())
```

Data Cleaning: Handling Missing Data

- Filling Missing Values

```
df['Age'] = df['Age'].fillna(0)
```

- Dropping Missing Values

```
df = df.dropna()
```

- Count Nulls per Columns

```
df.isnull().sum()
```

- Count duplicates for a specific column

```
df[clo_name'].duplicated().sum()
```

Advanced Features

- **Merging DataFrames**

```
df1 = pd.DataFrame({'ID': [1, 2], 'Name': ['Alice', 'Bob']})
df2 = pd.DataFrame({'ID': [1, 2], 'Age': [25, 30]})
merged = pd.merge(df1, df2, on='ID’)
```

- **Pivot Tables**

```
pivot = df.pivot_table(values='Age', index='City', aggfunc='mean')
```

.head():

Shows you the first 5 records in the data frame (optionally, if you want to see a different number of records, you can pass in a number)

.tail():

Same as head(), but it pull records from the end of the dataframe

.sample(n)

will give you a sample of *n* rows of the data -- just pass in a number

.info()
will give you a count of non-null values in each column.useful for seeing if any columns have null values
.describe()
will compute summary stats for numeric columns
.columns
will list the column names
.dtypes
will list the data types of each column
.shape
will give you a pair of numbers: *(number of rows, number of columns)*
nunique()
returns the total number of distinct, unique elements across a specified row or column.

Preprocessing for Machine Learning model

1. Handling Missing Values

Machine learning models cannot handle blank spaces or NaN values. You must either remove them or fill them.

A) Drop rows with any missing values

```
df = df.dropna()
```

B) Fill missing numerical values with the column mean

```
df["age"] = df["age"].fillna(df["age"].mean())
```

Preprocessing for Machine Learning model

2. Removing Duplicates

Duplicate rows do not provide new information and can artificially bias your model's training process.

A) Remove identical rows, keeping the first occurrence

```
df = df.drop_duplicates()
```

Preprocessing for Machine Learning model

3. Encoding Categorical Data

Machine learning models only understand math, so text categories must be converted into numerical values.

A) Convert text columns (e.g., 'City', 'Color') into 0s and 1s

```
df = pd.get_dummies(
        df,
        columns=["city", "color"],
        drop_first=True
        )
```

Preprocessing for Machine Learning model

4. Managing Outliers

Extreme values (outliers) skew model performance. You can cap them at specific percentiles using capping/winsorization.

A) Cap values below the 1st percentile and above the 99th percentile

```
lower, upper =
df["income"].quantile([0.01, 0.99])

df["income"] =
df["income"].clip(lower=lower, upper=upper)
```

Preprocessing for Machine Learning model

5. Feature Scaling (Normalization/Standardization)

If features have wildly different ranges (e.g., Age 0–100 vs. Salary 0–1,000,000), distance-based models will fail.

A) Min-Max Scaling: Shifts all values to scale precisely between 0 and 1

```
df["salary"] =
  (df["salary"] - df["salary"].min()) /
  (df["salary"].max() - df["salary"].min())
```

Preprocessing for Machine Learning model

6. Splitting Features and Targets

Before modeling, separate your independent variables (features, X) from your dependent variable (target, y).

A) X holds all columns except the target; y holds just the target column

```
X = df.drop(columns=["target_column"])
y = df["target_column"]
```

Additional Resources

- Pandas Documentation: https://pandas.pydata.org/docs/
- Practice Datasets: Kaggle Datasets
- Tutorials: Pandas Tutorials on w3schools, Real Python

- Cheat sheet: https://github.com/cjwinchester/pandas-cheat-sheet

https://www.w3schools.com/python/pandas/pandas_getting_started.asp